

AVL Tree Implementation

Complete Line-by-Line Code Explanation for Viva Preparation

Files Covered: **AVL.h** | **AVL.cpp** | **main.cpp**

Topics: Core AVL, Rotations, Traversals, Student Records, Leaf Ops, Successor/Predecessor, Level Queries, ReverseAVL

Table of Contents

Section	Topic
1	What is an AVL Tree? (Concept Overview)
2	AVL.h — Header File Explained
3	AVL.cpp — Core Functions (Constructor, Height, Balance Factor)
4	AVL.cpp — Rotations (Right, Left, balance())
5	Q1 — Insert & Delete (No Duplicates)
6	Q1 — Duplicate-Aware Insert & Delete
7	Q2 — Recursive Traversals (Inorder, Preorder, Postorder)
8	Q2 — Non-Recursive Traversals (Stack-Based)
9	Q3 — Reverse AVL Tree
10	Q4 — Student Record Insert & Search
11	Q5 — Leaf Nodes: Display & Delete
12	Q6 — Inorder Successor & Predecessor
13	Q7 — Nodes at a Given Level
14	Q8 — Display All Left-Child Nodes
15	Q9 — Largest Value at a Given Level
16	Q10 — Display All Right-Child Nodes
17	main.cpp — Menu Driver Explained
18	Likely Viva Questions & Answers

Section 1 — What is an AVL Tree?

An **AVL Tree** (named after Adelson-Velsky and Landis, 1962) is a self-balancing Binary Search Tree (BST). In a regular BST, insertions and deletions can degrade performance to $O(n)$ in the worst case (a skewed tree). An AVL tree solves this by maintaining a **balance factor** of -1, 0, or +1 at every node.

Key Properties

- Every node satisfies the BST property: left subtree keys < node key < right subtree keys.
- The **height** of each node is tracked. Height of a NULL node = 0; a single leaf has height 1.
- **Balance Factor (BF)** = height(left subtree) - height(right subtree). Must be -1, 0, or +1.
- If BF goes outside {-1, 0, 1} after an insert/delete, **rotations** are performed to restore balance.
- All insert, delete, and search operations run in **$O(\log n)$** time guaranteed.

The 4 Imbalance Cases and Their Fixes

Case	Trigger	Fix
LL (Left-Left)	$BF > 1$ and $BF(\text{left child}) \geq 0$	Single Right Rotation
LR (Left-Right)	$BF > 1$ and $BF(\text{left child}) < 0$	Left Rotation on child, then Right Rotation on node
RR (Right-Right)	$BF < -1$ and $BF(\text{right child}) \leq 0$	Single Left Rotation
RL (Right-Left)	$BF < -1$ and $BF(\text{right child}) > 0$	Right Rotation on child, then Left Rotation on node

Section 2 — AVL.h (Header File)

Include Guards

```
#ifndef AVL_H
#define AVL_H
...
#endif
```

#ifndef AVL_H — 'If Not Defined AVL_H': checks whether the macro AVL_H has been defined. If not, the contents of the file are included. This prevents the header from being included multiple times in the same translation unit, avoiding duplicate declaration errors.

#define AVL_H — defines the macro so subsequent includes are skipped.

#endif — ends the conditional block.

Standard Library Includes

```
#include
#include
#include
#include
```

<iostream> — Provides cin/cout for console input and output.

<stack> — Provides the STL stack used in non-recursive traversals (Section 8).

<string> — Provides std::string for the student name field.

<climits> — Provides INT_MIN, used as a sentinel value in largestAtLevel().

using namespace std; — Allows use of std library names (cout, string, stack) without the 'std::' prefix.

struct AVLNode — The Main Tree Node

```
struct AVLNode {
    int info; // Key value (also Roll No. in student mode)
    int count; // Duplicate counter
    int height; // Height of this node
    string name; // Student name
    float marks; // Student marks
    AVLNode *lchild; // Pointer to left child
    AVLNode *rchild; // Pointer to right child
};
```

info — The integer key stored in the node. In student mode, this doubles as the roll number.

count — How many times this value has been inserted. Used for duplicate-handling (Q1). For a fresh unique node, count = 1.

height — Cached height of this node's subtree. Updated after every insert/delete so rotations can read height in O(1) instead of traversing the tree.

name, marks — Extra data fields used only in Q4 (Student Record mode).

lchild, rchild — Raw pointers to left and right child nodes. NULL means no child.

struct RevAVLNode — Reverse AVL Node (Q3)

```
struct RevAVLNode {  
    int info;  
    int height;  
    RevAVLNode *lchild;  
    RevAVLNode *rchild;  
};
```

A simplified node for the ReverseAVL tree (Q3). It omits count/name/marks since those features are not needed there. The ordering rule is reversed: larger keys go left, smaller go right, so an inorder traversal yields values in descending order.

Class Declarations

class ReverseAVL — Encapsulates the reverse-ordered AVL tree. Public member 'root' is the entry point. Methods: height, balanceFactor, rotateRight, rotateLeft, insertRev, insert, inorder.

class AVLTree — The main AVL tree. Contains all Q1–Q10 functionality. Public member 'root' is the tree's root pointer. All methods are declared here and defined in AVL.cpp.

■ **Viva Tip:** Viva Tip: Know why we separate the declaration (AVL.h) from the definition (AVL.cpp). It allows multiple .cpp files to include the header without re-compiling the definitions. The linker then stitches them together.

Section 3 — Constructor, Height, Balance Factor

AVLTree Constructor

```
AVLTree::AVLTree() { root = NULL; }
```

This is the **constructor** — automatically called when an AVLTree object is created. It initialises the root pointer to NULL, meaning the tree starts empty. In main.cpp, the line **AVLTree tree;** triggers this constructor.

height() — Returns Cached Height

```
int AVLTree::height(AVLNode *n) {  
    return (n == NULL) ? 0 : n->height;  
}
```

Returns the stored height of node n. The ternary operator **(n == NULL) ? 0 : n->height** returns 0 for a NULL pointer (base case) and n->height otherwise. Height is cached in the node, so this runs in **O(1)**.

We need this helper because when a node is deleted or a child becomes NULL, we must still be able to query its height without crashing.

balanceFactor() — Computes BF at a Node

```
int AVLTree::balanceFactor(AVLNode *n) {  
    return (n == NULL) ? 0 : height(n->lchild) - height(n->rchild);  
}
```

Balance Factor = height(left) - height(right). If the result is:

- +2 or more — Left-heavy imbalance (LL or LR case needed)
- -2 or less — Right-heavy imbalance (RR or RL case needed)
- -1, 0, +1 — Balanced; no rotation needed

■ **Viva Tip:** Viva: Why store height in the node instead of computing it each time? Computing height by traversal would be $O(n)$ per operation, making insert/delete $O(n \log n)$ instead of $O(\log n)$.

Section 4 — Rotations and balance()

rotateRight() — Fixes Left-Left (LL) Imbalance

```
AVLNode *AVLTree::rotateRight(AVLNode *y) {
    AVLNode *x = y->lchild; // x becomes new root
    AVLNode *T2 = x->rchild; // T2 will be re-attached

    x->rchild = y; // y becomes right child of x
    y->lchild = T2; // T2 moves to y's left

    // Update heights – y first because it is now lower
    y->height = 1 + MAX(height(y->lchild), height(y->rchild));
    x->height = 1 + MAX(height(x->lchild), height(x->rchild));

    return x; // x is the new subtree root
}
```

Step-by-step: y is the unbalanced node (BF = +2). x is y's left child. T2 is x's right subtree. We rotate: x takes y's place, y becomes x's right child, T2 moves to y's left (preserving BST ordering). Heights are recalculated bottom-up.

Why update y before x? After the rotation, y is now a child of x. Its height must be correct before x's height can be computed.

rotateLeft() — Fixes Right-Right (RR) Imbalance

```
AVLNode *AVLTree::rotateLeft(AVLNode *x) {
    AVLNode *y = x->rchild;
    AVLNode *T2 = y->lchild;

    y->lchild = x;
    x->rchild = T2;

    x->height = 1 + MAX(height(x->lchild), height(x->rchild));
    y->height = 1 + MAX(height(y->lchild), height(y->rchild));

    return y;
}
```

Mirror image of rotateRight. y (right child) rises to become the new root. x sinks to y's left. T2 (y's old left child) moves to x's right to maintain BST property.

balance() — Detects and Applies the Correct Rotation

```
AVLNode *AVLTree::balance(AVLNode *n) {
    if (n == NULL) return NULL;
    n->height = 1 + MAX(height(n->lchild), height(n->rchild)); // Update height
    int bf = balanceFactor(n);

    // LL case
    if (bf > 1 && balanceFactor(n->lchild) >= 0) return rotateRight(n);
    // LR case
    if (bf > 1 && balanceFactor(n->lchild) < 0) {
```

```

n->lchild = rotateLeft(n->lchild); return rotateRight(n); }
// RR case
if (bf < -1 && balanceFactor(n->rchild) <= 0) return rotateLeft(n);
// RL case
if (bf < -1 && balanceFactor(n->rchild) > 0) {
n->rchild = rotateRight(n->rchild); return rotateLeft(n); }
return n; // Already balanced
}

```

Line 1: Height update — always done first so the BF is accurate.

LL Case: $BF > 1$ means left-heavy. Left child also left-heavy ($BF \geq 0$) → single right rotation.

LR Case: $BF > 1$ but left child is right-heavy ($BF < 0$). First rotate the left child left (straightening to LL), then rotate the node right.

RR Case: $BF < -1$, right child right-heavy → single left rotation.

RL Case: $BF < -1$, right child left-heavy → rotate right child right first, then rotate node left.

return n — If none of the four conditions triggered, n is balanced; return it unchanged.

■ **Viva Tip:** Viva: The LR and RL cases are called 'double rotations'. Interviewers often ask you to draw the before/after tree states on paper. Practice this.

Section 5 — Q1: Insert & Delete (No Duplicates)

insertNode() — Recursive BST Insert with Balancing

```
AVLNode *AVLTree::insertNode(AVLNode *node, int item) {
    if (node == NULL) { // Base case: create new node
        AVLNode *tmp = new AVLNode();
        tmp->info = item;
        tmp->count = 1;
        tmp->height = 1; // Leaf height is 1
        tmp->name = "";
        tmp->marks = 0.0f;
        tmp->lchild = NULL;
        tmp->rchild = NULL;
        return tmp;
    }
    if (item < node->info)
        node->lchild = insertNode(node->lchild, item);
    else if (item > node->info)
        node->rchild = insertNode(node->rchild, item);
    else { cout << "Item already present" << endl; return node; }
    return balance(node); // Rebalance on the way back up
}
```

Base case (node == NULL): We've found the correct empty slot. A new AVLNode is heap-allocated using 'new', all fields initialised, and returned. The caller assigns it as their child.

Recursive cases: If item < current node's info, go left. If item >, go right. The result is assigned back to node->lchild or node->rchild, which is how the new node gets linked into the tree.

Duplicate check: If item == node->info, print a message and return the node unchanged. No duplicate allowed.

balance(node): Called on every node as the recursion unwinds upward. Each ancestor updates its height and performs any needed rotation.

insert() — Public Interface

```
void AVLTree::insert(int item) {
    root = insertNode(root, item);
}
```

The public insert() simply calls the recursive insertNode() on the root and assigns the returned pointer back to root. This is important: if the root itself becomes unbalanced and a rotation occurs, the root pointer changes — which is why root must be reassigned.

minValueNode() — Finds Inorder Successor

```
AVLNode *AVLTree::minValueNode(AVLNode *node) {
    AVLNode *cur = node;
    while (cur->lchild != NULL) cur = cur->lchild;
    return cur;
}
```

Traverses left until there are no more left children. The leftmost node in any subtree holds the minimum value — this is the inorder successor when deleting a node with two children.

deleteNode() — Recursive Delete with Balancing

```
// If item < node->info: go left
// If item > node->info: go right
// Found node:
// Case A (leaf): delete node, return NULL
// Case B (1 child): copy child data into node, delete child
// Case C (2 children): replace with inorder successor data,
// then delete successor from right subtree
// Call balance(node) on the way back up
```

Case A — Leaf node: tmp = NULL. Free node memory, return NULL to detach from parent.

Case B — One child: tmp = the non-null child. Copy tmp's data into node (so node's address in the parent doesn't change), then delete tmp. Clever: avoids changing parent's pointer.

Case C — Two children: Find the inorder successor (smallest in right subtree). Copy its key/data into the current node. Then delete the successor from the right subtree recursively (which is simpler since the successor has at most one right child).

balance(node): Called again on each ancestor as recursion unwinds to maintain AVL property.

■ **Viva Tip:** Viva: Why use the inorder successor instead of predecessor for deletion? Either works. Successor (smallest in right subtree) is conventional and the code uses it consistently.

Section 6 — Q1: Duplicate-Aware Insert & Delete

The standard insert rejects duplicates. The duplicate-aware version instead maintains a **count** field in each node to track how many times a value has been inserted. This avoids creating multiple nodes with the same key — the tree structure stays valid.

insertDupNode() — Increments Count on Duplicate

```
if (item == node->info) {
    node->count++;
    cout << "Duplicate detected. Count for " << item
    << " is now " << node->count << endl;
    return node; // No structural change – no rebalancing needed
}
```

When a duplicate is found, we simply increment `node->count` and return immediately. Since no new node is added and the tree structure is unchanged, no rebalancing is needed.

deleteDupNode() — Decrements Count Before Physical Removal

```
if (node->count > 1) {
    node->count--;
    cout << "Duplicate removed. Count for " << item
    << " is now " << node->count << endl;
    return node; // Still has copies – don't delete node
}
// count == 1: fall through to normal node deletion logic
```

When `count > 1`, decrement and return — the node stays in the tree. Only when count reaches 1 do we proceed to the full deletion logic (same as `deleteNode` in Section 5).

■ **Viva Tip:** Viva: If you insert value 5 three times via `insertDup`, only one node exists in the tree with `count=3`. Deleting once brings count to 2. Deleting again brings it to 1. The third delete physically removes the node.

Section 7 — Q2: Recursive Traversals

All three traversals first check **if (root == NULL)** to handle the empty-tree case. Then they recurse on the non-null subtree rooted at ptr.

inorder() — Left, Root, Right (produces sorted output)

```
void AVLTree::inorder(AVLNode *ptr) {
    if (root == NULL) { cout << "Tree is empty"; return; }
    if (ptr != NULL) {
        inorder(ptr->lchild); // Visit left subtree first
        cout << ptr->info << " "; // Print current node
        inorder(ptr->rchild); // Visit right subtree last
    }
}
```

Because AVL (and all BSTs) maintain the property $\text{left} < \text{root} < \text{right}$, inorder traversal visits nodes in **ascending sorted order**.

preorder() — Root, Left, Right

```
cout << ptr->info << " "; // Root first
preorder(ptr->lchild);
preorder(ptr->rchild);
```

The root is printed before its subtrees. Useful for creating a copy of the tree or serialising it — you can rebuild the same BST by inserting in preorder sequence.

postorder() — Left, Right, Root

```
postorder(ptr->lchild);
postorder(ptr->rchild);
cout << ptr->info << " "; // Root last
```

Root is printed after both subtrees. Used when you need to process children before parents — for example, deleting all nodes (safe to delete a node only after its children are gone).

■ **Viva Tip:** Viva: What is the time complexity of recursive traversal? $O(n)$ — every node is visited exactly once. Space complexity is $O(h)$ for the call stack, where h = tree height = $O(\log n)$ for AVL.

Section 8 — Q2: Non-Recursive Traversals (Stack-Based)

The call stack used by recursion is replaced here by an explicit `std::stack`. This avoids stack-overflow on very deep trees and demonstrates manual stack management.

inorderNR() — Iterative Inorder

```
stack stk;
AVLNode *ptr = root;
while (ptr != NULL || !stk.empty()) {
    while (ptr != NULL) { stk.push(ptr); ptr = ptr->lchild; } // Push all lefts
    ptr = stk.top(); stk.pop();
    cout << ptr->info << " "; // Visit
    ptr = ptr->rchild; // Move to right subtree
}
```

Inner while: Keep pushing nodes and going left until we can't go further left.

stk.top() / pop(): Pop the topmost node (leftmost unvisited), print it, then move to its right subtree. This naturally produces Left → Root → Right order without recursion.

preorderNR() — Iterative Preorder

```
stk.push(root);
while (!stk.empty()) {
    AVLNode *ptr = stk.top(); stk.pop();
    cout << ptr->info << " ";
    if (ptr->rchild != NULL) stk.push(ptr->rchild); // Push right first
    if (ptr->lchild != NULL) stk.push(ptr->lchild); // Push left second
}
```

Push root. Each iteration: pop, print (Root), push right then left. Because stack is LIFO, left gets popped next, giving Root → Left → Right order.

Why push right before left? Stack pops the last-pushed item first. Pushing right before left ensures left is processed first.

postorderNR() — Iterative Postorder (Two Stacks)

```
stack stk1, stk2;
stk1.push(root);
while (!stk1.empty()) {
    AVLNode *ptr = stk1.top(); stk1.pop();
    stk2.push(ptr); // Collect in reverse
    if (ptr->lchild) stk1.push(ptr->lchild);
    if (ptr->rchild) stk1.push(ptr->rchild);
}
while (!stk2.empty()) { cout << stk2.top()->info << " "; stk2.pop(); }
```

Insight: Postorder is Left → Right → Root. If we reverse this: Root → Right → Left, that looks like a modified preorder. `stk1` produces Root → Right → Left order by pushing left before right (unlike preorder which pushes right before left). `stk2` collects this and when printed in reverse gives Left → Right → Root — correct postorder.

■ **Viva Tip:** Viva: Postorder NR is the trickiest. Explain that `stk2` reverses the Root-Right-Left order captured by `stk1`. Two-stack is the standard clean approach.

Section 9 — Q3: Reverse AVL Tree

A **Reverse AVL** is an AVL tree where the BST ordering is inverted: **larger values go LEFT, smaller values go RIGHT**. This means an inorder traversal (Left → Root → Right) visits larger values first, producing output in **descending order**.

insertRev() — Reversed Ordering Rule

```
// Reverse rule:
if (item > node->info)
    node->lchild = insertRev(node->lchild, item); // Larger goes LEFT
else
    node->rchild = insertRev(node->rchild, item); // Smaller goes RIGHT
```

The only difference from normal AVL insert is this reversed comparison. The rotation logic and height/balance factor calculations are identical.

inorder() on ReverseAVL — Descending Output

```
void ReverseAVL::inorder(RevAVLNode *ptr) {
    if (ptr != NULL) {
        inorder(ptr->lchild); // Visits LARGER values first
        cout << ptr->info << " ";
        inorder(ptr->rchild); // Then SMALLER values
    }
}
```

Since larger values are stored in the left subtree, visiting left first means the largest values are printed first — yielding descending order with no extra sorting step.

■ **Viva Tip:** Viva: What is the practical use of a Reverse AVL? Priority queues where the maximum element needs $O(\log n)$ access, or any scenario requiring sorted descending access.

Section 10 — Q4: Student Record Insert & Search

The same AVLTree is reused to store student records. The **info** field holds the Roll Number (integer key), while **name** (string) and **marks** (float) are satellite data that travel with the node.

insertStudentNode()

```
AVLNode *AVLTree::insertStudentNode(AVLNode *node, int rollNo,
const string &name;, float marks) {
    if (node == NULL) {
        AVLNode *tmp = new AVLNode();
        tmp->info = rollNo; // Key for BST ordering
        tmp->name = name;
        tmp->marks = marks;
        tmp->count = 1; tmp->height = 1;
        tmp->lchild = tmp->rchild = NULL;
        return tmp;
    }
    if (rollNo < node->info) node->lchild = insertStudentNode(node->lchild, rollNo, name,
marks);
    else if (rollNo > node->info) node->rchild = insertStudentNode(node->rchild, rollNo, name,
marks);
    else { cout << "Roll No already present"; return node; }
    return balance(node);
}
```

Structurally identical to insertNode(), but the new node also stores name and marks. BST ordering is still by roll number.

searchNode() — Recursive Search

```
AVLNode *AVLTree::searchNode(AVLNode *ptr, int key) {
    if (ptr == NULL) return NULL; // Not found
    if (key < ptr->info) return searchNode(ptr->lchild, key);
    if (key > ptr->info) return searchNode(ptr->rchild, key);
    return ptr; // Found
}
```

Standard BST search: if key is smaller go left, larger go right, equal means found. Returns a pointer to the node, or NULL if not found. Time complexity: $O(\log n)$.

searchStudent() — Displays Student Info

```
void AVLTree::searchStudent(int rollNo) {
    AVLNode *ptr = searchNode(root, rollNo);
    if (ptr == NULL) { cout << "Not found."; }
    else {
        cout << "Roll No: " << ptr->info << endl;
        cout << "Name : " << ptr->name << endl;
        cout << "Marks : " << ptr->marks << endl;
    }
}
```

■ **Viva Tip:** Viva: Why store student data in an AVL tree instead of a plain array? AVL tree gives $O(\log n)$ search by roll number. An unsorted array would be $O(n)$.

Section 11 — Q5: Leaf Nodes — Display & Delete

displayLeaves()

```
void AVLTree::displayLeaves(AVLNode *ptr) {
    if (root == NULL) { cout << "Tree is empty"; return; }
    if (ptr != NULL) {
        if (ptr->lchild == NULL && ptr->rchild == NULL)
            cout << ptr->info << " "; // It's a leaf – print it
        else {
            displayLeaves(ptr->lchild);
            displayLeaves(ptr->rchild);
        }
    }
}
```

A leaf is any node where both lchild and rchild are NULL. We recurse through the entire tree; whenever we hit a node with no children, we print it.

deleteLeavesHelper()

```
AVLNode *AVLTree::deleteLeavesHelper(AVLNode *ptr) {
    if (ptr == NULL) return NULL;
    if (ptr->lchild == NULL && ptr->rchild == NULL) {
        delete ptr; // Free memory
        return NULL; // Tell parent: this child no longer exists
    }
    ptr->lchild = deleteLeavesHelper(ptr->lchild);
    ptr->rchild = deleteLeavesHelper(ptr->rchild);
    return balance(ptr); // Re-balance after structural change
}
```

Post-order style: recurse into children first, then check if the current node is a leaf (which it might be after its children were removed). If it's a leaf, delete it and return NULL. `balance()` is called on the way back up to maintain AVL property.

■ **Viva Tip:** Viva: After deleting all leaves, the previous internal nodes may themselves become leaves. This function deletes only the leaves as they exist before the call — it does NOT iteratively remove new leaves created by deletions.

Section 12 — Q6: Inorder Successor & Predecessor

For a given key in the AVL tree:

- **Inorder Predecessor** = the largest value smaller than key = rightmost node in the left subtree.
- **Inorder Successor** = the smallest value larger than key = leftmost node in the right subtree.

findSuccessorPredecessor() — Single-Pass Algorithm

```
AVLNode *ptr = root, *successor = NULL, *predecessor = NULL;
while (ptr != NULL) {
    if (item == ptr->info) {
        // Found it: predecessor = rightmost of left subtree
        if (ptr->lchild != NULL) {
            AVLNode *tmp = ptr->lchild;
            while (tmp->rchild != NULL) tmp = tmp->rchild;
            predecessor = tmp;
        }
        // Successor = leftmost of right subtree
        if (ptr->rchild != NULL) {
            AVLNode *tmp = ptr->rchild;
            while (tmp->lchild != NULL) tmp = tmp->lchild;
            successor = tmp;
        }
        break;
    }
    else if (item < ptr->info) { successor = ptr; ptr = ptr->lchild; }
    else { predecessor = ptr; ptr = ptr->rchild; }
}
```

When traversing to find the key: If we go left ($\text{item} < \text{ptr}$), the current node ptr is a candidate successor (it's larger than item). If we go right ($\text{item} > \text{ptr}$), ptr is a candidate predecessor.

When key is found: Navigate directly to the rightmost of left subtree (predecessor) and leftmost of right subtree (successor).

■ **Viva Tip:** Viva: What if the key has no left subtree? Its predecessor is the last node from which we turned right while searching — tracked automatically by the traversal loop above.

Section 13 — Q7: Nodes at a Given Level

```
void AVLTree::nodesAtLevel(AVLNode *ptr, int target, int current) {  
    if (ptr == NULL) return;  
    if (current == target) { cout << ptr->info << " "; return; }  
    nodesAtLevel(ptr->lchild, target, current + 1);  
    nodesAtLevel(ptr->rchild, target, current + 1);  
}
```

Parameters: ptr = current node, target = desired level, current = current depth.

Base case 1: ptr == NULL — gone past a leaf, stop.

Base case 2: current == target — this node is at the desired level, print and stop recursing deeper.

Recursive step: Go into left and right children, incrementing current depth.

The root is level 1 (as called in main.cpp: nodesAtLevel(tree.root, level, 1)). Time complexity: $O(n)$ in the worst case (all nodes at the target level).

■ **Viva Tip:** Viva: This is essentially a depth-limited DFS. An alternative is BFS (level-order traversal using a queue) which would be more natural for printing all nodes at a level.

Section 14 — Q8: Display All Left-Child Nodes

```
void AVLTree::displayLeftNodes(AVLNode *ptr) {
    if (ptr == NULL) return;
    if (ptr->lchild != NULL) {
        cout << ptr->lchild->info << " "; // Print left child's value
        displayLeftNodes(ptr->lchild); // Recurse into left subtree
    }
    displayLeftNodes(ptr->rchild); // Also recurse right – to find ALL left-children
}
```

This function prints every node that is a left child of its parent (regardless of depth).

if (ptr->lchild != NULL): This node has a left child. Print that child's value, then recurse into the left subtree to find more left-children within it.

displayLeftNodes(ptr->rchild): Also recurse into the right subtree — there may be nodes there that are left-children of their own parents.

■ **Viva Tip:** Viva: The root is NOT printed because the root has no parent. Only nodes that are someone else's left child are printed. This is checked implicitly: we print ptr->lchild, not ptr itself.

Section 15 — Q9: Largest Value at a Given Level

```
int AVLTree::largestAtLevel(AVLNode *ptr, int target, int current) {
    if (ptr == NULL) return INT_MIN; // No node here – return lowest possible int
    if (current == target) return ptr->info; // This node is at target level

    int leftMax = largestAtLevel(ptr->lchild, target, current + 1);
    int rightMax = largestAtLevel(ptr->rchild, target, current + 1);

    return (leftMax > rightMax) ? leftMax : rightMax; // Return maximum
}
```

INT_MIN (from `<climits>`) is returned for NULL nodes. This acts as a sentinel: any real node value will be greater than **INT_MIN**, so the max comparison is always correct.

At the target level, we return the node's value directly. Otherwise we recurse both ways and return the maximum of the two subtrees at the target level.

In `main.cpp`: **if (result == INT_MIN) → 'No nodes at level'**. This happens when target is deeper than the tree.

■ **Viva Tip:** Viva: Why **INT_MIN** instead of -1 or 0? Because valid node values could be negative or zero. **INT_MIN** is guaranteed to be less than any realistic stored value.

Section 16 — Q10: Display All Right-Child Nodes

```
void AVLTree::displayRightNodes(AVLNode *ptr) {
    if (ptr == NULL) return;
    if (ptr->rchild != NULL) {
        cout << ptr->rchild->info << " "; // Print right child's value
        displayRightNodes(ptr->rchild); // Recurse into right subtree
    }
    displayRightNodes(ptr->lchild); // Also recurse left – to find ALL right-children
}
```

Mirror image of displayLeftNodes (Section 14). Prints every node that is a right child. Recurses into both subtrees to find all right-children throughout the whole tree.

Section 17 — main.cpp: Menu-Driven Driver

Object Creation

```
AVLTree tree;  
ReverseAVL revTree;
```

tree — An AVLTree object created on the stack. Its constructor (Section 3) runs automatically, setting root = NULL.

revTree — A ReverseAVL object for Q3 operations. Similarly initialised with root = NULL.

Main Loop

```
while (true) {  
    // print menu  
    cin >> choice;  
    switch (choice) { ... }  
}
```

An infinite loop keeps the program running until the user picks choice 6 (Quit), which calls **return 0**.

Key Switch Cases

- **Case 1 (Insert):** Reads an integer and calls tree.insert(value). Root is updated inside.
- **Case 2 (Delete):** Reads an integer and calls tree.deletion(value).
- **Case 3/4/5 (Traversals):** Calls inorder/preorder/postorder with tree.root as starting point.
- **Case 7 (NR Traversals):** A sub-menu with choices 1/2/3 selects inorderNR, preorderNR, postorderNR.
- **Case 9 (Students):** Sub-menu: sub=1 reads rollNo, name, marks and calls insertStudent; sub=2 calls searchStudent.
- **Case 14 (Largest at Level):** Calls largestAtLevel; checks if result == INT_MIN to report 'no nodes'.

cin.ignore() Before getline()

```
cin >> rollNo;  
cin.ignore(); getline(cin, sName);
```

After reading an integer with cin>>, a newline character '\n' remains in the input buffer. getline() would read that empty line instead of the actual name. **cin.ignore()** discards that leftover newline so getline() reads correctly.

■ **Viva Tip:** Viva: This is a common C++ gotcha. Always use cin.ignore() between cin>> and getline().

Section 18 — Likely Viva Questions & Answers

Q: What is an AVL tree and how does it differ from a BST?

A: An AVL tree is a self-balancing BST. Every node's balance factor ($\text{height}(\text{left}) - \text{height}(\text{right})$) is kept in $\{-1, 0, +1\}$. A regular BST can degrade to $O(n)$ for search/insert; AVL guarantees $O(\log n)$ always.

Q: What is a balance factor?

A: $\text{BF} = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$. If $|\text{BF}| > 1$, the node is unbalanced and a rotation is needed.

Q: Name the four rotation cases.

A: LL (right rotation), RR (left rotation), LR (left then right), RL (right then left).

Q: Why do we update the child's height before the parent's in rotations?

A: After a rotation, the child is now lower in the tree. Its height must be correct before the parent (now higher) can compute its own height from its children.

Q: What is the time complexity of AVL insert/delete/search?

A: $O(\log n)$ for all three, because the tree height is always $O(\log n)$.

Q: How does duplicate handling work in this code?

A: The count field in AVLNode tracks how many times a key has been inserted. On duplicate insert, count is incremented; on delete, decremented. Physical removal only happens when count reaches 1.

Q: How does non-recursive inorder work?

A: Use an explicit stack. Push nodes going left until NULL, then pop, print, and move right. Repeat until stack is empty and ptr is NULL.

Q: Why does postorder NR use two stacks?

A: The first stack produces a Root-Right-Left traversal. The second stack reverses it to Left-Right-Root (correct postorder).

Q: What is a Reverse AVL?

A: An AVL where larger values go LEFT and smaller go RIGHT. Inorder traversal then produces values in descending order.

Q: What is the inorder successor of a node?

A: The node with the smallest key greater than the given key. Found as the leftmost node in the right subtree.

Q: What is the inorder predecessor?

A: The node with the largest key smaller than the given key. Found as the rightmost node in the left subtree.

Q: Why use `new` for node allocation?

A: Heap allocation with 'new' persists beyond the function's scope. Stack-allocated variables are destroyed when the function returns — unsuitable for tree nodes.

Q: What does `#ifndef` do in the header file?

A: It's an include guard. It prevents the header contents from being processed more than once in a single translation unit, avoiding duplicate declarations.

Q: Why assign `root = insertNode(root, item)` in `insert()`?

A: If a rotation occurs at the root level, the root pointer changes. Assigning the return value ensures the tree's root pointer always reflects the current actual root.

Q: What is INT_MIN used for?

A: It's a sentinel value from representing the smallest possible integer. Used in `largestAtLevel()` as the default for NULL nodes, ensuring any real value beats it in a max comparison.

End of Document — Good Luck in Your Viva!